

ZVOTE: DECENTRALIZED ELECTRONIC VOTING SYSTEM

Comprehensive Technical Reference & Master Project Defense Manual

Target System:	University SRC & Departmental Elections
Core Architecture:	Hybrid Web2.5 (PostgreSQL + Polygon EVM Smart Contracts)
Smart Contract:	ZVote.sol (Solidity 0.8.19) on Polygon Amoy (Chain ID: 80002)
Live Contract Address:	0xFf9d890459593d883a2D3BA024b566017958dfFb
Live Platforms:	Frontend: Netlify (SPA) Backend: Render.com DB: Neon PostgreSQL
Prepared For:	Final Year Engineering Project Defense & Examination Committee

1. Executive Summary & Problem Statement

Democratic legitimacy in student governance rests upon the unquestionable integrity of elections. However, university elections across institutions face recurring crises of trust due to inherent vulnerabilities in existing voting modalities:

Modality	Critical Vulnerabilities & Failure Modes
Paper Ballots	Vulnerable to physical ballot-box stuffing, ballot destruction, recount disputes, chain-of-custody contamination, logistical delay, and high administrative print/stationery costs.
Traditional Electronic Voting (Web2)	Relies on centralized SQL databases (MySQL/PostgreSQL) where database administrators or rogue insiders can execute unauthorized queries (e.g. UPDATE candidates SET votes = votes + 500) with zero public cryptographically provable detection.
The Secrecy Paradox	Traditional systems either reveal who voted for whom to prove tally authenticity, or hide identities so completely that voters cannot verify if their individual ballot was altered or discarded.
Pure Web3 Voting	Requires every voter to own a Web3 crypto wallet (MetaMask), hold native cryptocurrency (MATIC/POL) to pay gas fees, and understand blockchain transactions, making universal student adoption impossible.

The ZVote Solution (Hybrid Web2.5): ZVote solves this trilemma by combining Web2 user convenience with Web3 cryptographic immutability. Students authenticate with their familiar Institutional Student ID and password with \$0 gas fees (no MetaMask needed). In the background, the backend acts as an automated cryptographic relay, submitting and sealing each ballot onto an immutable EVM smart contract on the Polygon blockchain.

2. System Architecture & Zero-Linkability Model

ZVote is engineered upon a decoupled two-tier architecture that enforces mathematical voter privacy:

Layer	Component & Technology	Data Stored & Cryptographic Responsibilities
Off-Chain Layer (Web2)	Node.js, Express, Prisma ORM, Neon PostgreSQL (AWS)	• Stores student identities (Name, Student ID, Email, Department). • Enforces eligibility and departmental enclaves. • Records VoteReceipt (User ID, Position ID, txHash) to prevent duplicate submissions. • NEVER stores or logs which candidate the student selected.
On-Chain Layer (Web3)	Solidity 0.8.19, Polygon Amoy Testnet (Proof of Stake)	• Canonical ballot box and state machine. • Stores position titles, candidate rosters, and immutable tallies (voteCount++). • Enforces votedForPosition[positionId][voterAddress] = true. • NEVER receives student names, emails, or index numbers.
Relayer Layer	Ethers.js v6 with NonceManager	• Signs blockchain transactions on behalf of voters using the Election Authority wallet. • Manages sequential nonces to prevent race conditions during peak voting bursts. • Pays gas fees so voters incur \$0 cost.
Messaging Layer	Brevo HTTPS REST API (Port 443)	• Bypasses cloud host outbound SMTP port restrictions (ports 25, 465, 587). • Dispatches credentials directly to external student Gmail inboxes. • Rate-limited intermittent queue protects domain reputation.

Mathematical Principle of Zero Linkability: An attacker who gains complete, unrestricted read access to both the PostgreSQL database AND the public Polygon blockchain ledger cannot link a specific student ID to their chosen candidate. In PostgreSQL, only eligibility and transaction hashes are recorded. On Polygon, only candidate vote tallies and pseudonymous wallet addresses are recorded. The mapping between voter identity and ballot selection is destroyed in transient memory during request processing.

3. Technology Stack & Technical Justifications

Examiners frequently ask: *'Why did you choose this specific technology over its alternatives?'* Below are the rigorous technical justifications:

- **Solidity 0.8.19 (Smart Contracts):** Features built-in arithmetic overflow and underflow protection without needing SafeMath. Compiled with 200 optimization runs to reduce runtime gas execution cost on EVM chains. Emits standardized events (VoteCast, ElectionStarted) for transparent public log scraping.
- **Polygon Amoy (PoS / CDK):** Why not Ethereum Mainnet? Ethereum takes 12-15 seconds per block and charges \$5 to \$50 in gas per transaction—prohibitive for university elections. Polygon provides 2-second block finality, 65,000 TPS capacity, and transaction fees of fractions of a cent (\$0.0001). Why not Solana? Polygon is 100% EVM-compatible,

enabling standard Hardhat and OpenZeppelin tooling.

- **Node.js & Express API:** Single-threaded event loop with non-blocking asynchronous I/O handles thousands of concurrent student requests without thread-pool starvation. Seamlessly integrates with the JavaScript-native Ethers.js v6 blockchain SDK.
- **Ethers.js v6 & NonceManager:** Provides modern BigInt support, modular contract interaction, and automatic JSON-RPC failover. Wrapped with NonceManager to solve the critical 'nonce too low' and 'replacement fee too low' race conditions that occur when an election authority wallet signs multiple votes in quick succession.
- **Prisma ORM & Neon PostgreSQL:** PostgreSQL enforces strict ACID compliance and foreign key constraints across elections, positions, and voter audit receipts. Neon provides serverless branching, auto-scaling, and secure SSL pooling over AWS us-east-2.
- **Brevo HTTPS REST API:** Cloud hosting providers (Render, AWS EC2, Vercel) permanently block outbound SMTP ports 25, 465, and 587 on free tiers to prevent spam. Nodemailer over SMTP hangs and times out. Brevo's REST API operates over standard HTTPS (Port 443), ensuring 100% reliable credential delivery to external Gmail inboxes without requiring a custom domain.
- **React 18 & Vite:** Component-based virtual DOM ensures instant UI state transitions. Vite leverages native ES modules and Rollup for a lightning-fast build time (<3 seconds) and a production bundle under 320 KB, ensuring fast mobile loading even on low-bandwidth campus cellular networks.
- **TailwindCSS:** Utility-first styling avoids bloated runtime CSS. Provides full mobile-first responsiveness with fluid grid breakpoints, flexbox layout, and custom accessibility attributes.

4. Smart Contract Architecture & On-Chain Logic

The smart contract ZVote.sol deployed at 0xFf9d890459593d883a2D3BA024b566017958dfFb serves as the immutable single source of truth for voting tallies:

Contract Element	Implementation & Security Guarantee
electionAuthority	Immutable state variable set to msg.sender upon deployment. Protected by onlyElectionAuthority modifier. Restricts administrative functions (create position, add candidate, start/close election) to the verified backend wallet.
electionActive	Boolean gate. When false, all calls to vote() immediately revert with 'ZVote: election is not active', preventing early voting or post-deadline ballot tampering.
votedForPosition	Nested mapping uint256 => mapping(address => bool). Tracks whether a specific voter wallet address has voted in a specific position. If true, subsequent vote attempts revert with 'ZVote: voter has already voted for this position'.
vote() Function	Atomically increments candidates[_candidateId].voteCount++ and sets votedForPosition[_positionId][_voter] = true. Emits an indexed VoteCast(voter, positionId, candidateId) event on-chain.
getResults()	Read-only view function returning parallel arrays of candidate names, programmes, and vote counts. Invoked by the frontend results dashboard once polls are closed.

5. Cryptographic Security & Anti-Fraud Mechanisms

ZVote enforces defensive security across every layer of the election lifecycle:

- **Double-Voting Immunity (Two-Tier Guard):** Tier 1 (Off-Chain): PostgreSQL uniqueness constraint on (userId, positionId) in VoteReceipt table. If a student tries to vote twice, the DB query fails with HTTP 409 Conflict. Tier 2 (On-Chain): The smart contract checks require(!votedForPosition[posId][voter]). Even if the off-chain database were hacked, the blockchain mathematically rejects duplicate votes.
- **Voter Password Security & Administrator Blindness:** When temporary passwords are generated, they are hashed with Bcrypt (salt rounds = 10) for database authentication. Before returning registration reports to the administrator, the backend transforms the passwords into one-way SHA-256 hashes (e.g. sha256:7f83b...). The administrator cannot read student passwords.
- **Forced Password Change on First Login:** Student accounts are created with mustChangePassword = true. Upon initial sign-in, the UI forces the student to set a private password before ballot access is granted.
- **Session Invalidation via SERVER_BOOT_ID:** Every time the backend boots, it generates an ephemeral in-memory UUID. All issued JWTs carry this bootId. If the server is restarted or redeployed, all existing tokens instantly invalidate, preventing replay attacks.
- **Candidate Exclusivity & Enclave Filtering:** A student cannot run for multiple positions in the same election. In departmental elections, candidate and voter queries enforce strict department filtering so voters cannot cross into other faculties.
- **Sealed Ballot Protocol (Mitigating Bandwagon Bias):** While polls are open, candidate vote counts are sealed in the frontend and only real-time voter turnout is displayed. This eliminates psychological bandwagon bias and tactical voter coercion.

6. Constitutional Voting Algorithms & Business Logic

ZVote implements standard university constitutional electoral formulas directly in code:

1. The 50% + 1 Absolute Majority Formula:

Let total valid votes cast for a position be V .

The Constitutional Majority Threshold is defined as: $T = \lceil V / 2 \rceil + 1$

- **Case A (WON_MAJORITY):** Top candidate votes $\geq T \rightarrow$ Declared Elected Winner.
- **Case B (RUNOFF_REQUIRED):** Top candidate votes $< T \rightarrow$ System flags Runoff Election between top two contenders.
- **Case C (TIE):** Top two candidates receive identical vote tallies $\rightarrow$ System flags Runoff Election.

2. Unopposed Candidates (YES / NO Referendum):

When only one candidate contests a position, ZVote configures a democratic Referendum. Voters must choose **YES (Approve)** or **NO (Reject)**. The candidate is declared elected if and only if: **YES votes > 50% of total referendum ballots cast.**

7. Master Project Defense Q&A;: Top 25 Questions & Answers

Below are the 25 most critical questions external examiners and faculty professors ask during project defenses, accompanied by precise, high-scoring technical responses:

Q1: What motivated the development of ZVote when online voting systems already exist?

Traditional electronic voting systems rely on centralized databases (MySQL, MongoDB, PostgreSQL) where system administrators have god-mode privileges to manipulate election results using direct SQL updates with no verifiable public audit trail. ZVote integrates blockchain smart contracts to decentralize the ballot box, ensuring that once a vote is cast, it is immutable, tamper-proof, and publicly verifiable.

Q2: Why did you choose Polygon Amoy rather than Ethereum Mainnet or a local blockchain?

Ethereum Mainnet is impractical for university elections due to high transaction gas fees (\$5–\$50 per vote) and slow block times (12–15 seconds). A local Hardhat node proves code locally but provides zero public verifiability. Polygon Amoy PoS testnet offers 2-second block times, high throughput (65,000 TPS), EVM compatibility, and sub-cent gas fees, allowing us to deploy a real, live, publicly verifiable system.

Q3: How do you protect voter secrecy if the blockchain is public and transparent?

Through our Zero-Linkability decoupled architecture. Personal student demographic information (Name, Student ID, Email) lives exclusively in our off-chain PostgreSQL database. The smart contract on the blockchain only receives pseudonymous candidate indices and an abstract voter address. The link between student identity and candidate choice is never written to disk anywhere.

Q4: If voters don't have MetaMask or cryptocurrency, who pays the gas fees?

ZVote uses a Gasless Relayer Pattern. The Election Authority wallet signs and submits all vote transactions on behalf of the voters, absorbing the minor gas cost (~0.001 POL per vote). Students simply authenticate with their familiar student ID and password, requiring zero crypto knowledge.

Q5: What prevents the Election Authority / Relayer from altering the student's vote?

When the student submits their vote, the backend immediately returns the cryptographic transaction receipt containing the mined transaction hash (`txHash`). The student can click 'Verify on Polygonscan' or use the in-app Vote Verification portal to view the raw blockchain receipt, which confirms the exact position ID and candidate index sealed in the block.

Q6: How does the system prevent a student from voting twice?

Through a two-tier guard. Tier 1 (Off-Chain): The PostgreSQL `VoteReceipt` table has a composite unique constraint on `(userId, positionId)`. If a student attempts a second vote, the database query aborts. Tier 2 (On-Chain): The Solidity contract maintains a mapping `votedForPosition[positionId][voter]`. If a duplicate vote reaches the contract, the transaction reverts with an error.

Q7: Why did you use Brevo HTTPS API instead of standard SMTP like Gmail or Sendgrid?

Cloud platform free tiers (Render, AWS EC2, Vercel) block outbound TCP ports 25, 465, and 587 by default to prevent spam relays. Standard SMTP connections hang and time out. Brevo's REST API operates over standard HTTPS port 443, which is universally open, ensuring 100% reliable credential dispatch directly to students' personal inboxes.

Q8: Can the election administrator see student passwords?

No. When student accounts are generated, passwords are encrypted using Bcrypt (10 salt rounds) for database verification. For administrative views and CSV exports, the backend transforms the passwords into one-way cryptographic SHA-256 hashes. The plain text password is only dispatched to the student's personal email.

Q9: What happens if a position only has one candidate?

ZVote automatically triggers an Unopposed YES/NO Approval Referendum. The voter selects either YES (Approve) or NO (Reject). The candidate must achieve greater than 50% YES votes of total ballots cast to be declared elected.

Q10: What is your 50% + 1 victory calculation and how does it handle ties?

We calculate the majority threshold as $T = \text{floor}(V / 2) + 1$, where V is the total votes cast. If the leading candidate meets or exceeds T , they win (`WON_MAJORITY`). If no candidate reaches T , or if the top two candidates receive identical votes, the system classifies the result as `RUNOFF_REQUIRED`.

Q11: Why are vote tallies hidden while the election is active?

To uphold the Sealed Ballot Protocol and eliminate bandwagon bias. In behavioral political science, displaying real-time vote totals influences undecided voters and encourages tactical voting. ZVote displays live voter turnout percentage while keeping candidate totals encrypted and sealed until polls officially close.

Q12: How do you handle network race conditions during high-volume voting bursts?

We wrapped the ethers.js signing wallet in a `NonceManager`. In Ethereum, if multiple transactions are sent concurrently with the same nonce, the transaction fails with 'nonce too low'. The `NonceManager` automatically queues and increments nonces sequentially, ensuring zero failed transactions during peak voting hours.

Q13: How does ZVote handle departmental election restrictions?

Through Departmental Enclaves. When creating a departmental election, the administrator assigns a `targetDepartment`. The backend and frontend filter candidates and voters so that only students whose registered department matches the election target department can view and vote on that ballot.

Q14: What is the purpose of SERVER_BOOT_ID in the JWT authentication?

Whenever the backend server restarts or is redeployed, it generates a fresh UUID called `SERVER_BOOT_ID`. When a user presents a JWT, the server verifies that the token's `bootId` matches the active server session. This provides instant global session revocation upon server restart.

Q15: How are candidate pictures handled?

Administrators can either supply an external HTTPS picture URL or upload candidate portraits directly via the `uploadImage` endpoint. The server validates image MIME types (JPEG, PNG, WebP) and enforces a 2MB file size ceiling to prevent denial-of-service through storage exhaustion.

Q16: How scalable is the system for a campus of 20,000 students?

The off-chain API is stateless and can scale horizontally across multiple instances behind a load balancer. PostgreSQL read-replicas handle voter profile queries. On Polygon, 20,000 votes represent ~0.3% of the network's daily throughput, well within standard operational capacity.

Q17: What prevents an attacker from voting after the election end time?

Both off-chain and on-chain checks prevent late voting. The API validates `new Date() <= election.endTime`. In addition, when the administrator closes the election, the backend calls `endElection()` on the smart contract, which sets `electionActive = false`. Any subsequent on-chain vote attempt reverts immediately.

Q18: What is the role of Prisma ORM in the system?

Prisma generates type-safe database queries and migration scripts, preventing SQL injection vulnerabilities. It establishes foreign key cascades between elections, positions, candidates, and vote receipts.

Q19: How did you optimize gas consumption in your smart contract?

We used Solidity 0.8.19 optimizer set to 200 runs, packed struct fields compactly, used `uint256` for native EVM word alignment, and replaced costly on-chain string lookups with compact integer indices for positions and candidates.

Q20: Can a student vote on their smartphone?

Yes. ZVote is fully responsive, built with TailwindCSS mobile-first design, fluid grid layouts, and an animated mobile navigation drawer. Students can authenticate and vote from any iOS or Android browser.

Q21: What happens if a student forgets their password?

The student clicks 'Forgot Password' on the login screen and enters their email. The backend creates a cryptographic 32-byte hex token with a 1-hour expiration timestamp in the database and dispatches a password reset link via Brevo.

Q22: Why not store student personal info on IPFS (InterPlanetary File System)?

IPFS is a public, immutable, distributed content-addressed network. Storing personally identifiable student information (PII) on IPFS violates global data privacy regulations (GDPR, Data Protection Act) because records on IPFS cannot be modified or deleted. PII must remain in an off-chain relational database.

Q23: How does the system handle bulk student roster ingestion?

The system uses `xlsx` library to parse `.csv` or `.xlsx` files into normalized objects. It performs data sanitization, checks for duplicates against the database, generates unique wallet addresses and temporary passwords, and intermittently queues welcome emails to prevent provider rate-limiting.

Q24: What is individual verifiability versus universal verifiability?

Individual Verifiability: A student can verify that their specific ballot was included in the blockchain ledger using their transaction hash. Universal Verifiability: Any observer, auditor, or candidate can independently inspect the smart contract's public tallies and sum the votes to confirm the accuracy of the final winner without needing privileged credentials.

Q25: What is the final conclusion and recommendation of this project?

ZVote successfully proves that blockchain technology can eliminate election tampering in university governance without imposing technical friction or financial cost on voters. The hybrid Web2.5 architecture represents the optimal paradigm for institutional electronic voting.